

Controle de Estoque com Login e Senha + SQLite

Contexto

Você foi contratado para desenvolver um sistema interno de controle de estoque.

Somente usuários cadastrados poderão acessar o sistema.

O aplicativo deverá utilizar **Flutter + SQLite** para armazenar usuários e produtos.

Antes de começar

O projeto será executado no **Google Chrome**.

Adicione as dependências utilizadas na aula no `pubspec.yaml`.

O professor fornecerá dois arquivos:

```
None
sqlite3.wasm
sqflite_sw.js
```

Coloque os dois arquivos dentro da pasta:

```
None
web/
```

A estrutura deverá ficar semelhante a:

```
None
web/
├─ index.html
├─ sqlite3.wasm
└─ sqflite_sw.js
```

Execute o projeto utilizando:

```
Shell  
flutter run -d chrome --web-port 8080
```

Use sempre a porta 8080 durante os testes.

Objetivo

O aplicativo deverá possuir:

```
None  
CADASTRO  
  ↓  
SQLite  
  
LOGIN  
  ↓  
Consulta no SQLite  
  
ACESSO LIBERADO  
  ↓  
CONTROLE DE ESTOQUE
```

Tela 1 - Login

Ao abrir o aplicativo:

```
None  
ÁREA RESTRITA  
  
E-mail:  
[-----]  
  
Senha:
```

[_____]

[ENTRAR]

Ainda não possui conta?

[CRIAR CONTA]

Tela 2 - Cadastro

O usuário deverá informar:

None

Nome

E-mail

Senha

Ao pressionar:

None

CADASTRAR

o usuário deverá ser salvo no SQLite.

Depois do cadastro, volte para a tela de login.

Banco de dados - Usuários

Crie uma tabela:

None

usuarios

com:

None

id

nome

email

senha

Exemplo:

id	nome	email	senha
1	Lucas	lucas@email.com	1234
2	Ana	ana@email.com	abc123

Login

Quando o usuário clicar em:

None

ENTRAR

o aplicativo deverá verificar se existe no banco um usuário com:

None

email informado

E

senha informada

Fluxo:

None

Usuário digita

↓

Provider

↓
Service
↓
SQLite
↓
SELECT
↓
Usuário encontrado?

Se o login estiver correto

O usuário deverá acessar a tela principal.

Exemplo:

None
Olá, Lucas!

CONTROLE DE ESTOQUE

[VER PRODUTOS]

[CADASTRAR PRODUTO]

[SAIR]

O nome mostrado deverá ser o nome encontrado no SQLite.

Se o login estiver incorreto

Mostre:

None
E-mail ou senha incorretos.

Tela 3 - Cadastro de Produto

O usuário deverá conseguir cadastrar produtos utilizando:

None

Nome

Categoria

Quantidade

Preço

Exemplo:

None

Nome :

Teclado

Categoria:

Informática

Quantidade:

5

Preço :

250.00

Ao clicar:

None

CADASTRAR PRODUTO

o produto deverá ser salvo no SQLite.

Banco de dados - Produtos

Crie também a tabela:

```
None
produtos
```

com:

```
None
id
nome
categoria
quantidade
preço
```

Exemplo:

id	nome	categoria	quantidade	preço
1	Teclado	Informática	5	250.00
2	Mouse	Informática	2	120.00

Tela 4 - Estoque

Os produtos cadastrados deverão aparecer em uma lista.

Exemplo:

```
None
ESTOQUE

Teclado
Informática
Quantidade: 5
R$ 250,00

-----

Mouse
```

Informática
Quantidade: 2
R\$ 120,00

ESTOQUE BAIXO

Quando:

None
quantidade <= 3

mostre:

None
ESTOQUE BAIXO

Alterar quantidade

Cada produto deverá possuir:

None
[-] Quantidade: 5 [+]

Ao pressionar +:

None
5 → 6

Ao pressionar -:

None
5 → 4

A alteração deverá ser salva no SQLite.

Não basta alterar apenas o valor mostrado na tela.

Excluir produto

Adicione uma opção:

```
None
EXCLUIR
```

Ao excluir um produto, ele deverá:

```
None
ser removido do SQLite
      ↓
desaparecer da lista
```

Acompanhar o banco pelo terminal

Além de executar as operações no SQLite, o aplicativo deverá utilizar `print()` para mostrar no terminal o que aconteceu.

INSERT

Depois de cadastrar um usuário:

```
None
print('Usuário cadastrado: $nome');
```

Exemplo no terminal:

```
None
INSERT → Usuário cadastrado: Maria
```

Depois de cadastrar um produto:

None

INSERT → Produto cadastrado: Teclado

SELECT

Ao realizar o login, mostre no terminal se o usuário foi encontrado.

Exemplo:

None

SELECT → Procurando usuário: maria@email.com

SELECT → Usuário encontrado: Maria

ou:

None

SELECT → Usuário não encontrado

Ao carregar o estoque:

None

SELECT → Produtos encontrados:

{id: 1, nome: Teclado, quantidade: 5}

{id: 2, nome: Mouse, quantidade: 2}

UPDATE

Ao alterar a quantidade:

None

UPDATE → Teclado

Quantidade anterior: 5

Nova quantidade: 6

DELETE

Ao excluir:

None

DELETE → Produto excluído: Teclado

Função para visualizar produtos do banco

Crie também uma função que consulte todos os produtos e imprima o resultado.

Exemplo:

None

```
Future<void> mostrarProdutosNoTerminal() async {  
    final db = await banco;  
  
    final produtos = await db.query('produtos');  
  
    print('==== PRODUTOS NO BANCO ====');  
  
    for (final produto in produtos) {  
        print(produto);  
    }  
}
```

O objetivo é conseguir verificar pelo terminal se os dados realmente foram salvos no SQLite.

Exemplo:

None

```
==== PRODUTOS NO BANCO ====  
{id: 1, nome: Teclado, categoria: Informática, quantidade: 5,  
preco: 250.0}  
{id: 2, nome: Mouse, categoria: Informática, quantidade: 2,  
preco: 120.0}
```

Operações utilizadas

Durante a atividade serão utilizadas as quatro principais operações do banco:

None

Cadastrar usuário

→ INSERT

Realizar login

→ SELECT

Cadastrar produto

→ INSERT

Mostrar produtos

→ SELECT

Alterar quantidade

→ UPDATE

Excluir produto

→ DELETE

Testes

Teste 1 - Login

Cadastre:

None

Nome: Maria

E-mail: maria@email.com

Senha: 1234

Depois faça login utilizando esses dados.

Confira também o resultado mostrado no terminal.

Teste 2 - Login incorreto

Tente:

None

E-mail: maria@email.com

Senha: 9999

O sistema deverá negar o acesso.

No terminal deverá aparecer algo semelhante a:

None

SELECT → Usuário não encontrado

Teste 3 - Estoque

Cadastre pelo menos três produtos.

Um deles deverá possuir:

None

quantidade <= 3

e deverá aparecer como:

None

ESTOQUE BAIXO

Confira também os produtos impressos no terminal.

Teste 4 - UPDATE

Escolha um produto e altere sua quantidade utilizando:

None

[-] e [+]

Verifique no terminal a quantidade anterior e a nova quantidade.

Depois recarregue a página e confirme que a nova quantidade continua salva.

Teste 5 - DELETE

Exclua um produto.

Ele deverá desaparecer da tela.

Confira também no terminal:

None

DELETE → Produto excluído

Teste 6 - Persistência

Depois de cadastrar o usuário e os produtos:

1. pressione **F5**;
2. faça login novamente;
3. abra o estoque.

Os usuários, produtos e quantidades deverão continuar cadastrados.

Regra importante

Não permita dois usuários com o mesmo e-mail.

Caso isso aconteça, mostre:

None

E-mail já cadastrado.

Antes de realizar o INSERT, será necessário consultar o banco para verificar se o e-mail já existe.

None

SELECT

↓

E-mail existe?

↓

SIM → Não cadastrar

NÃO → INSERT

Organização sugerida

None

lib/

|

├─ main.dart

|

├─ models/

|

├─ usuario.dart

|

└─ produto.dart

|

├─ services/

|

├─ usuario_service.dart

|

└─ produto_service.dart

|

├─ providers/

|

├─ usuario_provider.dart

|

└─ produto_provider.dart

|

└─ pages/

|

├─ login_page.dart

|

├─ cadastro_page.dart

|

├─ home_page.dart

|

├─ cadastro_produto_page.dart

|

└─ estoque_page.dart

Resultado esperado

O aplicativo deverá permitir:

None

Cadastrar usuário



Fazer login



Entrar no sistema



Cadastrar produtos



Visualizar estoque



Alterar quantidade



Excluir produtos

Todos os dados deverão ser armazenados no **SQLite**.

Durante a execução, as operações realizadas no banco também deverão aparecer no terminal:

None

INSERT

SELECT

UPDATE

DELETE